

W13D02 — SQL DDL & Schema Design

PostgreSQL · CREATE TABLE · Constraints · Schema untuk ML System

1. Apa itu DDL?

DDL (Data Definition Language) adalah subset SQL yang digunakan untuk **mendefinisikan struktur database** — bukan untuk mengisi data (itu DML), melainkan untuk membuat tabel, kolom, dan aturan mainnya. Statement utama: CREATE TABLE, ALTER TABLE, DROP TABLE.

Analogi: DDL adalah cetak biru gudang logistik. Sebelum paket bisa masuk, kamu harus mendefinisikan rak mana yang ada, label apa yang wajib ditempel, dan aturan penolakan apa yang berlaku — bukan oleh karyawan (aplikasi), tapi oleh sistem gudang itu sendiri.

2. Constraints — Penjaga Integritas Data

Constraints adalah aturan yang di-enforce langsung oleh PostgreSQL engine — bukan oleh aplikasi. Ini penting karena aplikasi bisa punya bug, bisa di-bypass, atau bisa crash. Database tidak.

Prinsip Defense in Depth: Data masuk ke database bisa lewat 3 jalur — FastAPI (yang punya Pydantic), script admin, atau psql langsung saat debugging. Pydantic hanya jaga pintu nomor 1. Constraints di database menjaga semua pintu sekaligus.

Constraint	Fungsi	Null OK?	Contoh
PRIMARY KEY	Identitas unik tiap baris. Otomatis UNIQUE + NOT NULL.	Tidak	id SERIAL PRIMARY KEY
NOT NULL	Kolom wajib diisi — database tolak INSERT tanpa nilai.	Tidak	email TEXT NOT NULL
UNIQUE	Nilai tidak boleh duplikat. Null tetap boleh (bahkan banyak empty).	Tidak	email TEXT UNIQUE
FOREIGN KEY	Nilai harus ada di tabel lain. Cegah orphan records.	Tidak	REFERENCES models(id)
CHECK	Validasi kondisi kustom. Baris gagal kondisi tidak.	Tidak	CHECK (score BETWEEN 0 AND 1)
DEFAULT	Nilai otomatis jika tidak diisi. Bukan validasi.	Ya	DEFAULT NOW()

3. Tipe Data Penting untuk ML Schema

Tipe	Keterangan	Kapan pakai
SERIAL	Auto-increment integer. PostgreSQL kelola otomatis.	ID internal, tidak di-expose ke client
UUID	Universally Unique Identifier. Tidak sequential.	ID yang di-expose ke client / API response
TIMESTAMPTZ	Timestamp WITH timezone. Konsisten lintas timezone.	Selalu pakai ini, bukan TIMESTAMP biasa
NUMERIC(p,s)	Angka presisi EXACT. p=total digit, s=digit desimal.	Confidence score, harga, nilai kritis
FLOAT	Angka presisi APPROX. Ada floating point error.	Jangan pakai untuk nilai kritis production
JSONB	Binary JSON. Lebih cepat di-query dari JSON biasa.	Input data ML yang strukturnya tidak tetap

INET	Tipe khusus IP address (IPv4 & IPv6). Divalidasi DBClient IP, source address logging	
TEXT	String panjang tidak terbatas.	Semua string umum

NUMERIC vs FLOAT: FLOAT menyimpan $0.1 + 0.2$ sebagai 0.30000000000000004 — acceptable untuk ML training, berbahaya untuk financial/confidence score di production. Selalu pakai NUMERIC untuk nilai yang harus exact.

4. Schema ML Prediction System

Dua tabel yang membentuk fondasi sistem logging prediksi ML: **models** (katalog model yang di-deploy) dan **predictions** (setiap prediction request yang masuk ke API).

Tabel: models

```
CREATE TABLE models ( id SERIAL PRIMARY KEY, name TEXT NOT NULL, version TEXT NOT NULL, CONSTRAINT uq_model_name_version UNIQUE (name, version), created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() );
```

CONSTRAINT uq_model_name_version UNIQUE (name, version) — Composite unique constraint. Boleh ada 'fraud_detector' v1.0 dan v2.0, tapi tidak boleh dua 'fraud_detector' v1.0. Nama constraint eksplisit memudahkan debugging saat error muncul di log.

Tabel: predictions

```
CREATE TABLE predictions ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), model_id INTEGER NOT NULL REFERENCES models(id) ON DELETE RESTRICT, input_data JSONB NOT NULL, prediction TEXT NOT NULL, confidence NUMERIC(5,4) NOT NULL CHECK (confidence >= 0.0 AND confidence <= 1.0), latency_ms INTEGER NOT NULL CHECK (latency_ms >= 0), status TEXT NOT NULL DEFAULT 'success' CHECK (status IN ('success', 'error', 'timeout')), client_ip INET, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() );
```

Baris	Kenapa penting
UUID DEFAULT gen_random_uuid()	ID tidak sequential — tidak bisa ditebak, aman di-expose ke client lewat API
ON DELETE RESTRICT	Model tidak bisa dihapus jika masih ada prediksi yang merujuknya — audit trail aman
JSONB NOT NULL	Input data fleksibel (schema bisa berubah tiap model) tapi keberadaannya wajib
NUMERIC(5,4)	Presisi exact untuk confidence score — tidak ada floating point error
CHECK (status IN (...))	Enum validation tanpa tipe data terpisah — DB tolak nilai di luar whitelist
INET (nullable)	Tipe khusus IP — DB validasi format. Nullable karena tidak selalu tersedia

5. Relasi Antar Tabel & Index

Relasi antara **models** dan **predictions** adalah one-to-many: satu model bisa punya nol atau banyak predictions, tapi setiap prediction pasti punya tepat satu model.

Index dibuat pada kolom yang sering di-query:

```
CREATE INDEX idx_predictions_model_id ON predictions(model_id); CREATE INDEX idx_predictions_created_at ON predictions(created_at);
```

Tanpa index, query 'tampilkan semua prediksi model X' harus scan seluruh tabel (full table scan). Dengan index, PostgreSQL langsung lompat ke baris yang relevan. Index pada **created_at** penting untuk monitoring: 'prediksi error 1 jam terakhir?'

6. ON DELETE: RESTRICT vs CASCADE

	ON DELETE RESTRICT	ON DELETE CASCADE
Perilaku	Database TOLAK DELETE pada parent jika masih ada data yang merujuk ke parent.	Data yang merujuk ke semua child secara otomatis saat parent dihapus.
Gunakan untuk	Data yang tidak boleh hilang: audit trail, log transaksi, data yang tidak bermakna tanpa parent: session tokens, temp files.	
Risiko	DELETE gagal — harus hapus child manual dulu.	Data hilang permanen tanpa peringatan eksplisit.
ML Use Case	GUNAKAN INI. Prediksi harus tetap ada meskipun model dihapus.	Gunakan ini untuk menghapus predictions — audit trail harus intact.

7. Modifikasi Schema dengan ALTER TABLE

Schema bukan sesuatu yang sekali jadi. Kebutuhan berubah — kolom baru perlu ditambah, tipe data perlu diubah. ALTER TABLE handle ini tanpa harus drop dan recreate tabel.

```
-- Tambah kolom baru (nullable, tanpa default) ALTER TABLE predictions ADD COLUMN
client_ip INET; -- Tambah kolom dengan default ALTER TABLE predictions ADD COLUMN
retry_count INTEGER NOT NULL DEFAULT 0; -- Ubah tipe data kolom (hati-hati di
production!) ALTER TABLE predictions ALTER COLUMN prediction TYPE VARCHAR(500); --
Tambah constraint setelah tabel dibuat ALTER TABLE predictions ADD CONSTRAINT
chk_latency CHECK (latency_ms >= 0);
```

Catatan production: ALTER TABLE pada tabel besar (jutaan baris) bisa menyebabkan lock. Operasi seperti ADD COLUMN dengan DEFAULT di PostgreSQL 11+ sudah aman (instant). Tapi ALTER COLUMN TYPE masih memerlukan full table rewrite.

8. Command Referensi

```
-- Copy file schema ke container docker cp schema.sql pg-server:/tmp/schema.sql --
Jalankan file SQL docker exec -it pg-server psql -U postgres -d postgres -f
/tmp/schema.sql -- Masuk psql interaktif docker exec -it pg-server psql -U postgres
-d postgres -- Di dalam psql: \dt -- list semua tabel \d predictions -- describe
struktur tabel + constraints + index \d models \q -- keluar psql
```

9. Knowledge Check — Poin Kunci

Pertanyaan	Jawaban
UNIQUE + NULL: boleh banyak null?	Ya. NULL dianggap 'unknown', bukan nilai. Dua unknown tidak bisa dibandingkan sebagai sama. Untuk ML, NULL bisa berarti "tidak diketahui" atau "tidak relevan".
NUMERIC(5,4) vs FLOAT?	NUMERIC adalah exact number — tidak ada floating point error. FLOAT adalah approximate. Gunakan NUMERIC untuk perhitungan keuangan atau ilmiah yang presisi. Untuk ML, FLOAT biasanya lebih cepat dan cukup akurat.
ON DELETE RESTRICT vs CASCADE?	RESTRICT: tolak DELETE parent jika ada child. CASCADE: hapus child otomatis. Untuk ML audit trail, gunakan RESTRICT. Untuk menghapus model dan predictions, gunakan CASCADE.
Kenapa UUID, bukan SERIAL untuk predictions?	UUID adalah non-sequential — tidak bisa ditebak urutan ID berikutnya. Aman di-expose ke client. SERIAL adalah sequential dan bisa ditebak.
Kenapa constraints di DB, bukan hanya di aplikasi?	DB constraints bisa di-bypass (script admin, psql langsung, bug). DB constraints berlaku untuk semua jalur akses data.